

P1865R0

Add max() to latch and barrier

Published Proposal, 2019-09-29

This version:

<https://wg21.link/P1865R0>

Authors:

[David Olsen](#) (NVIDIA)

[Olivier Giroux](#) (NVIDIA)

Audience:

LEWG, LWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

- 1 Abstract
- 2 Motivation
- 3 Wording
 - 3.1 class `latch`
 - 3.2 class `barrier`

§ 1. Abstract

Add a `static constexpr ptrdiff_t max() noexcept;` function to classes `latch` and `barrier`, which returns the maximum number of threads that the implementation supports arriving at the `latch` or `barrier`.

§ 2. Motivation

Both `latch` (`[thread.latch]`) and `barrier` (`[thread.barrier]`) have a constructor that takes a `ptrdiff_t expected` parameter that indicates how many threads need to arrive at the latch or barrier before waiting threads are unblocked. The constructors in both classes require that `expected` be non-negative. But they don't place any upper limit on the value of the argument. It is well-formed for a program to create a latch or barrier with an expected count of `std::numeric_limits<ptrdiff_t>::max()`. It is unlikely that an implementation could support that in a useful way.

Implementations of `latch` and `barrier` can have better performance without sacrificing any usefulness if the implementor is allowed to limit the expected count.

§ 3. Wording

§ 3.1. class `latch`

Change the beginning of the class summary in 32.8.3 "Class `latch`" [`thread.latch.class`] as follows:

```
namespace std {
    class latch {
    public:
        static constexpr ptrdiff_t max() noexcept;

        constexpr explicit latch(ptrdiff_t expected);
        // ...
```

Insert the following after paragraph 2 (just before the constructor) in [`thread.latch.class`]:

```
static constexpr ptrdiff_t max() noexcept;

Returns: The maximum value of counter that the implementation supports.
```

Change the description of the constructor in [`thread.latch.class`] as follows:

```
constexpr explicit latch(ptrdiff_t expected);
```

Expects: `expected >= 0` is true and `expected <= max()` is true.

Effects: Initializes counter with `expected`.

Throws: Nothing.

§ 3.2. class `barrier`

Change the beginning of the class summary in 32.8.4.2 "Class template `barrier`" [**thread.barrier.class**] as follows:

```
namespace std {  
    template<class CompletionFunction = see below>  
    class barrier {  
    public:  
        using arrival_token = see below;  
  
        static constexpr ptrdiff_t max() noexcept;  
  
        constexpr explicit barrier(ptrdiff_t expected,  
                                   CompletionFunction f = CompletionFunction  
        // ...
```

Insert the following just after paragraph 7 (just before the constructor) in [**thread.barrier.class**]:

static constexpr ptrdiff_t max() noexcept;

Returns: The maximum expected phase count that the implementation supports.

Change the description of the constructor in [**thread.latch.class**] as follows:

```
constexpr explicit barrier(ptrdiff_t expected,  
                           CompletionFunction f = CompletionFunction());
```



Expects: `expected >= 0` is true and `expected <= max()` is true.

Effects: Sets both the initial expected count for each barrier phase and the current expected count for the first phase to `expected`. Initializes completion with `std::move(f)`. Starts the first phase. [*Note:* If `expected` is 0 this object can only be destroyed. -- *end note*]

Throws: Any exception thrown by `CompletionFunction`'s move constructor.